

# Análise e Design de Software e o Processo Unificado

Profa. Dra. Lina Garcés

# Paradigma de Desenvolvimento de Software OO

## PRINCÍPIOS

### Abstração

Módulos, componentes, classes.

### Encapsulamento

Ocultamento de informações e operações, Facilidade de mudança



### Herança

Reuso de estrutura de dados e de comportamento . Coesão.

### Polimorfismo

Flexibilidade na definição de métodos. Facilidade de evolução (incremento)

**MODELAGEM** - Diagramas UML (Unified Modeling Language)

**PROCESSO DE DESENVOLVIMENTO DE SOFTWARE** - Etapas de Análise, Design e Implementação

# O que aprenderá na aula?

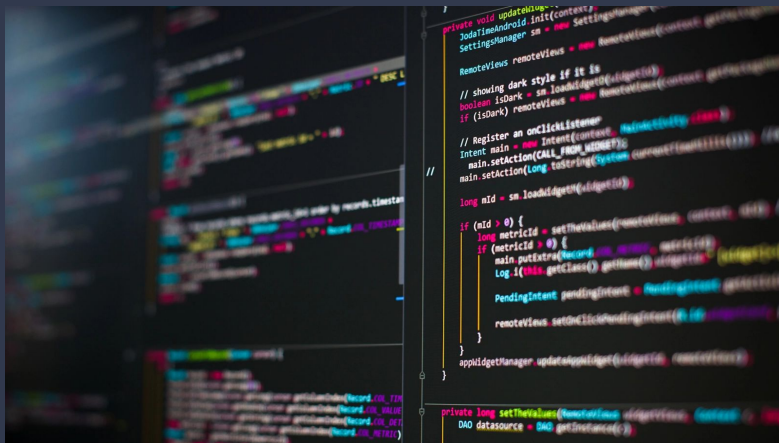
- Definição de Sistema de Software e Como este é Desenvolvido
- Processo Unificado e as Atividades de Análise e Projeto
- Modelagem nos Sistemas de Software
- Resumo



O que é Software?

Como o Software é criado?

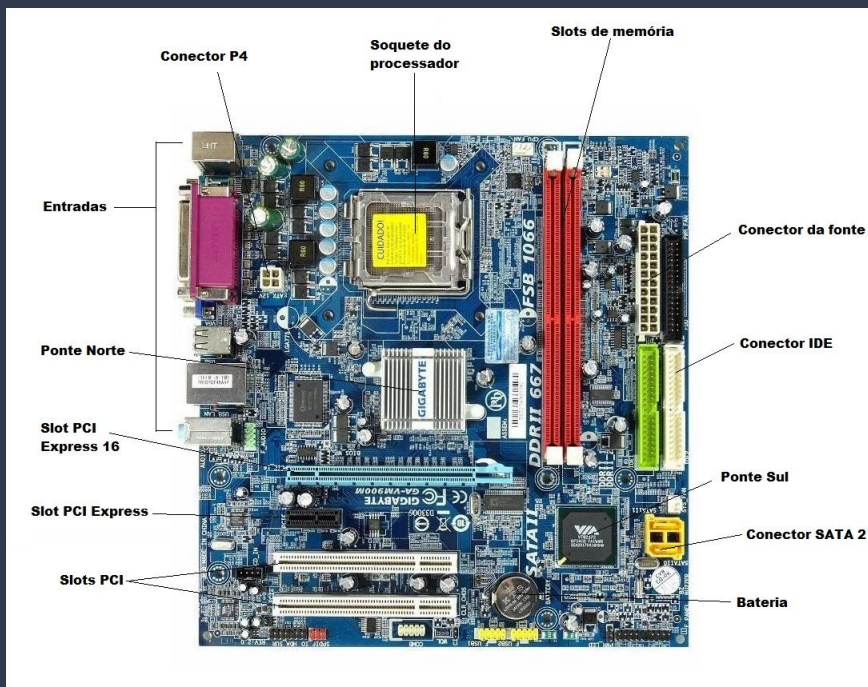
# O Software **é mais** que só Código Fonte



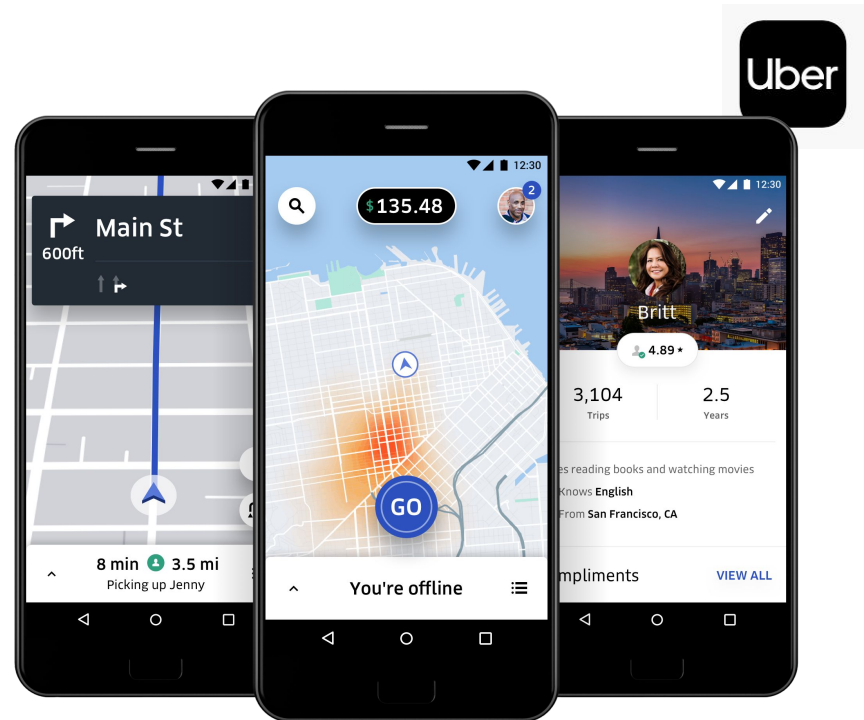
Software consiste de:

- **Instruções**, que quando executadas produzem a função e o desempenho desejados
- **Estruturas de dados**, que possibilitam que os programas manipulem adequadamente a informação
- **Documentos**, que descrevem a operação e o uso dos programas

# A maioria das vezes, o Software é FEITO SOB MEDIDA



Hardware é construído **ensamblando** componentes existentes



Software pode **reusar** componentes existentes, mas precisa de **desenvolvimento** sob medida

# Software não é Fabricado é DESENVOLVIDO através de um Processo de Engenharia (Descoberta e Criatividade)

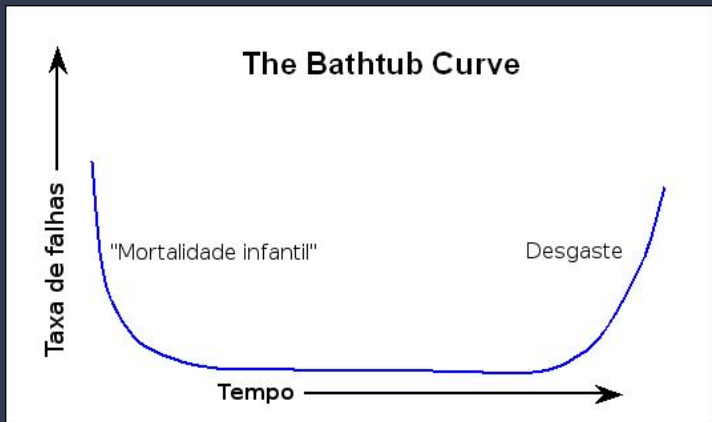


Linha de Fabricação de Hardware

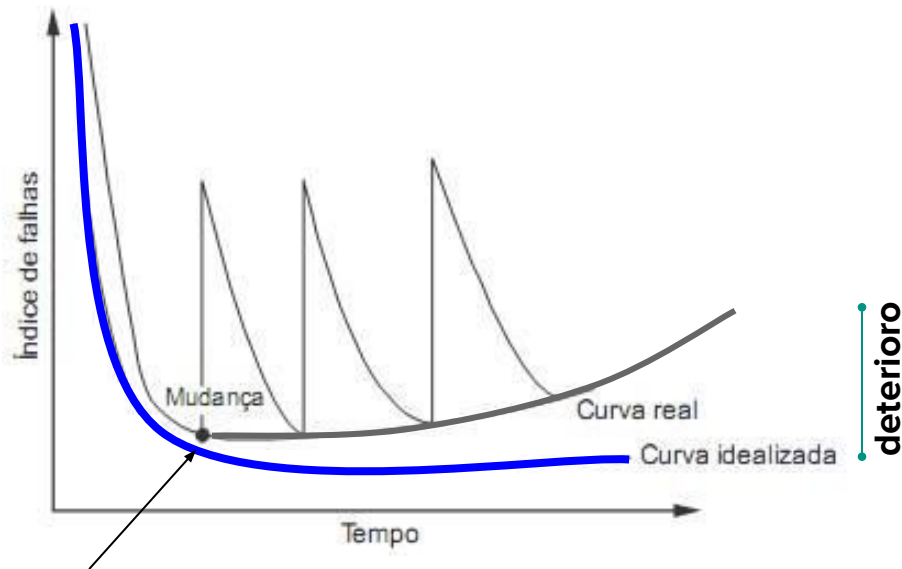


Desenvolvimento de Software

# Software não se desgasta, se **DETERIORA**



Curva de Defeitos para Hardware



Curva de Defeitos para Software

**Manutenção de Hardware é mais fácil que a manutenção de Software**

# Processo de Desenvolvimento de Software (PDS)



Conjunto de **atividades relacionadas** que levam ao **desenvolvimento** de um **produto de software**.

Para **cada atividade**, o PDS define:

- **Produtos** gerados em cada atividade
- **Papéis** dos responsáveis envolvidos em cada atividade
- **Pré e pós-condições** das atividades (entradas/saídas)

# Modelos de PDS

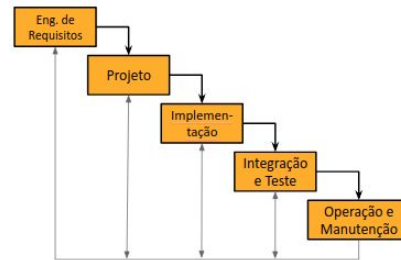
Representação simplificada dos PDS

## Modelos Tradicionais

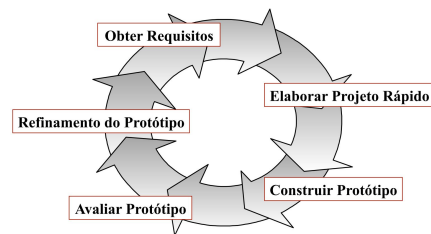
1. Modelo Cascata/Linear
2. Prototipação
3. Espiral
4. Incremental
5. RUP (Rational Unified Process)

## Modelos Ágeis

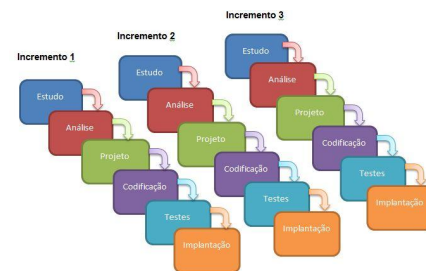
1. SCRUM
2. XP
3. Crystal
4. Kanban
5. Lean



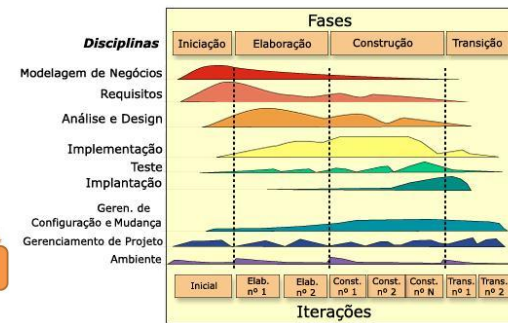
Cascata



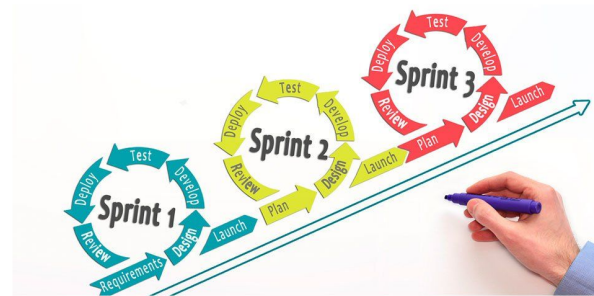
Prototipação



Incremental



RUP



SCRUM

# Discussão

- Benefícios de usar PDS?
- Quais as diferenças entre os PDS apresentados?
- Qual o melhor PDS?

# Quer conhecer mais sobre PDS ...

- Processo em Cascata: <https://www.youtube.com/watch?v=5A5XCuWMG4o>
- Processo em Espiral: <https://www.youtube.com/watch?v=mp22SDTnsQQ>
- Prototipação:  
<https://www.youtube.com/watch?v=bAEnaGG8Otc&list=PLAwxTw4SYaPkNAtqsKcFkUGpf4j67NBaz&index=29>
- RUP  
<https://www.youtube.com/watch?v=YgkhFH8g0J4&list=PLAwxTw4SYaPkNAtqsKcFkUGpf4j67NBaz&index=30>
- SCRUM: <https://www.youtube.com/playlist?list=PLRj4UESgCLz1uwaFje483eV8hQiBguW-P>
- XP (eXtreme Programming): <https://www.youtube.com/watch?v=hbFOwqYIOcU>
- Processos ágeis: <https://www.youtube.com/watch?v=WjwEh15M5Rw>

# Por que usar PDS?

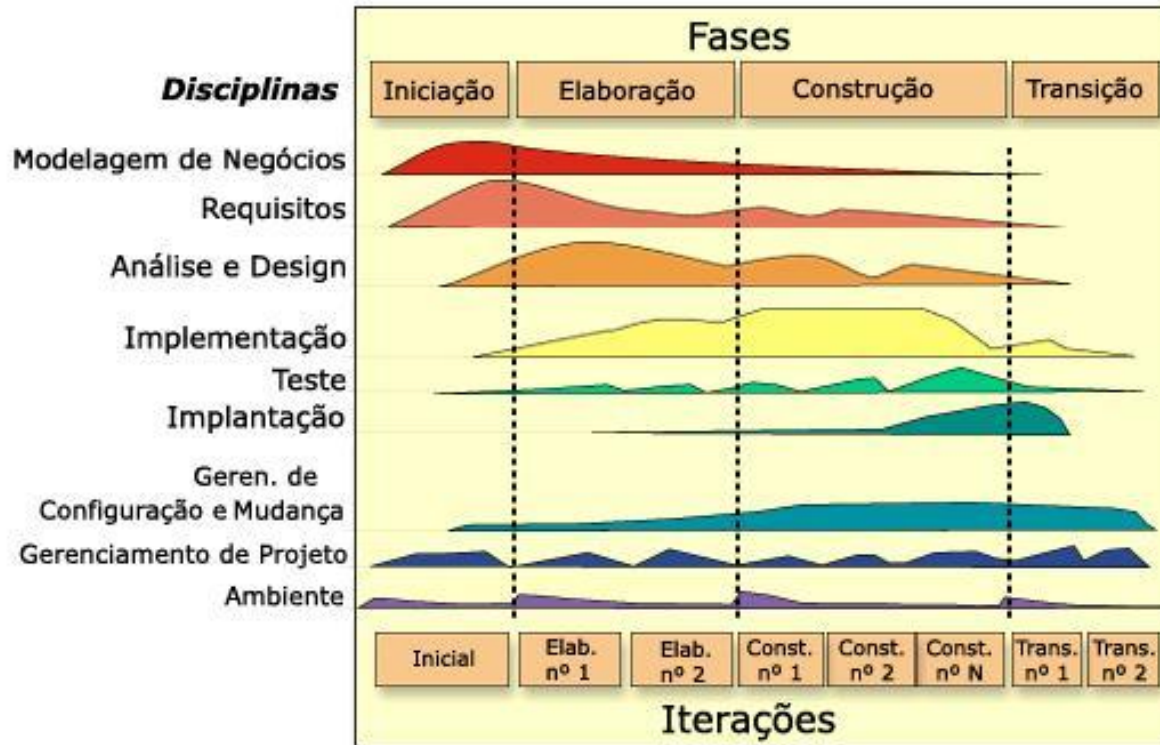
Evitar consequências negativas para os clientes, usuários finais e outros stakeholders



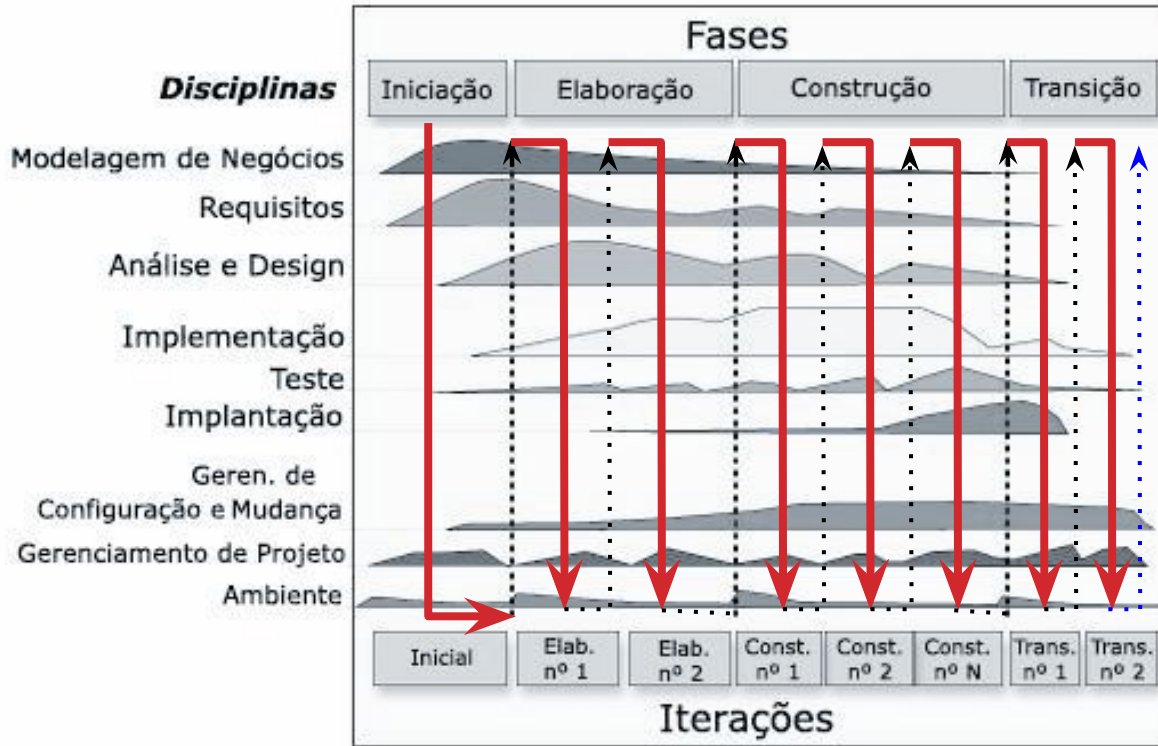
- Software de difícil manutenção (correção, evolução)
  - Alto custo de manutenção
  - Dificuldade para corrigir erros
  - Dificuldade para evoluir o software
- Dificuldade de reutilizar código
- Código mal elaborado e propagação de erros
- Sistemas com baixa performance, segurança, disponibilidade, escalabilidade, etc.
- Falta de confiança nos dados apresentados pelo sistema
- Software difícil de usar, baixa satisfação do usuário
- fracasso do projeto
- prejuízo à empresa que desenvolve o software
- entre muitos outros problemas ...

# Processo Unificado de Desenvolvimento de Software

# Processo Unificado



# Processo Unificado – É Iterativo e Incremental



# As Disciplinas do Processo Unificado

**Modelagem  
de Negócios**

**Requisitos**

**Análise e  
Projeto  
(Design)**

**Codificação**

**Teste e  
Integração**

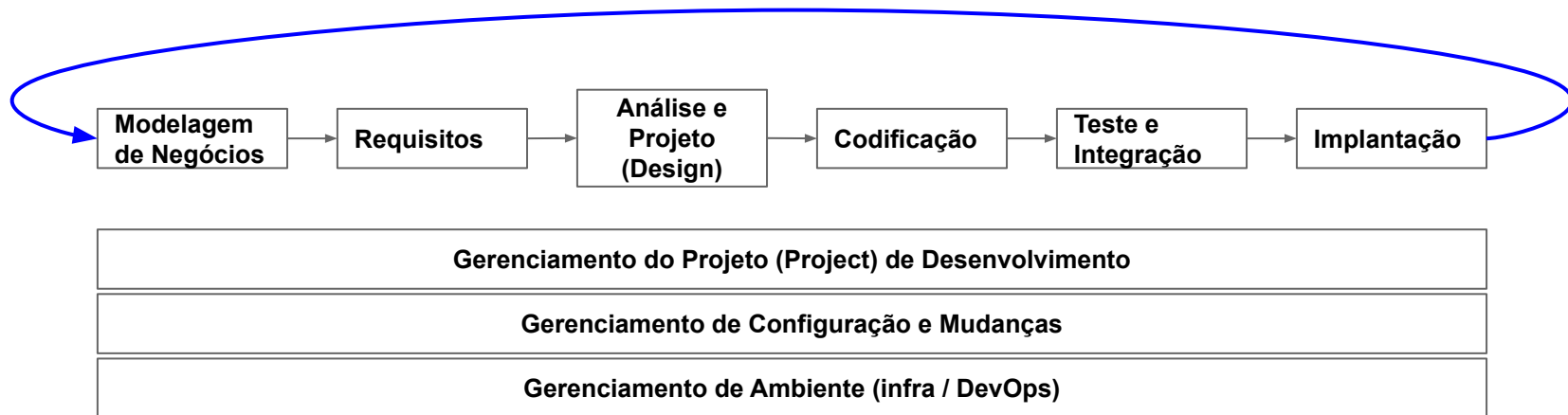
**Implantação**

**Gerenciamento do Projeto (Project) de Desenvolvimento**

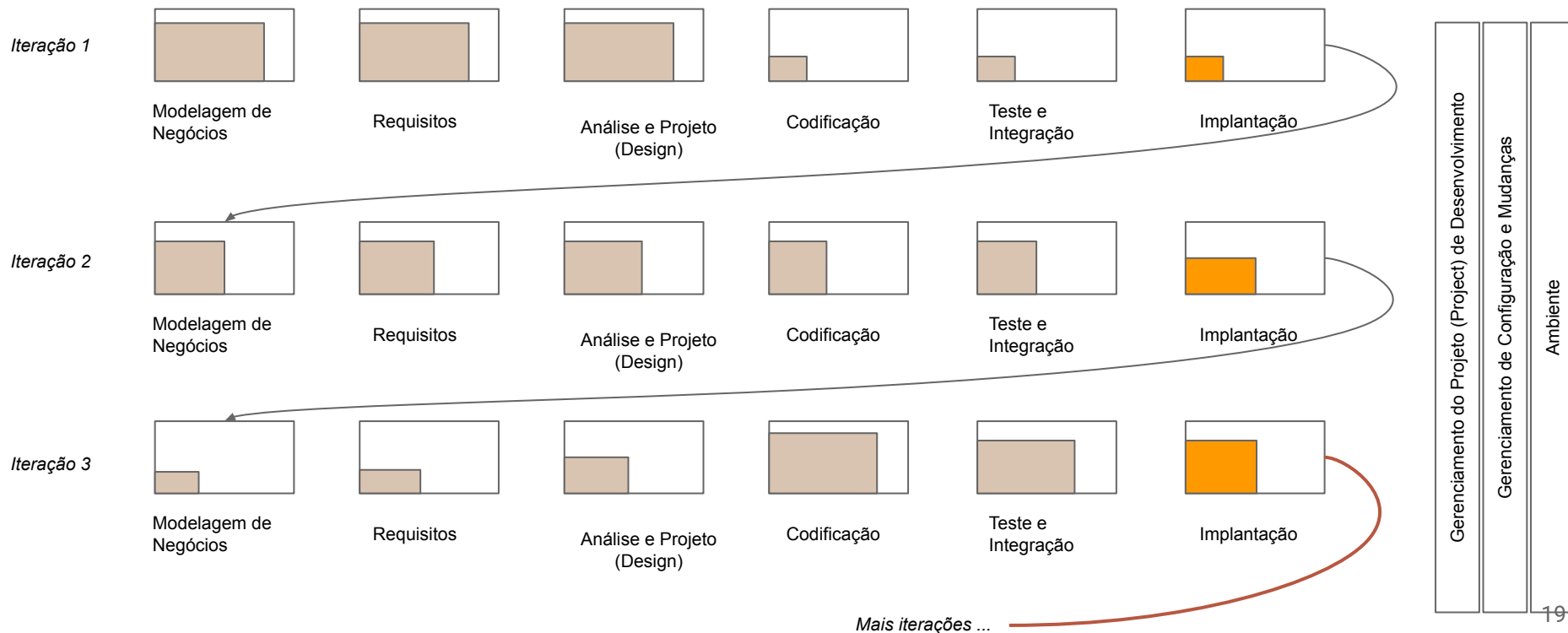
**Gerenciamento de Configuração e Mudanças**

**Gerenciamento de Ambiente (infra / DevOps)**

# O Processo Unificado é Iterativo



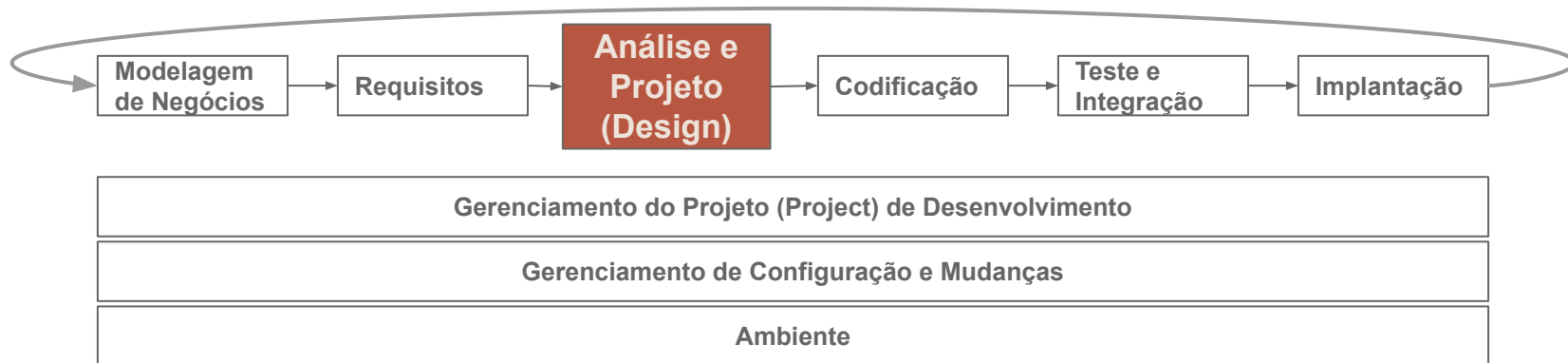
# O Processo Unificado é Iterativo e Incremental



# Análise e Projeto (*Design*) de Software

**Nosso foco em APOO!**

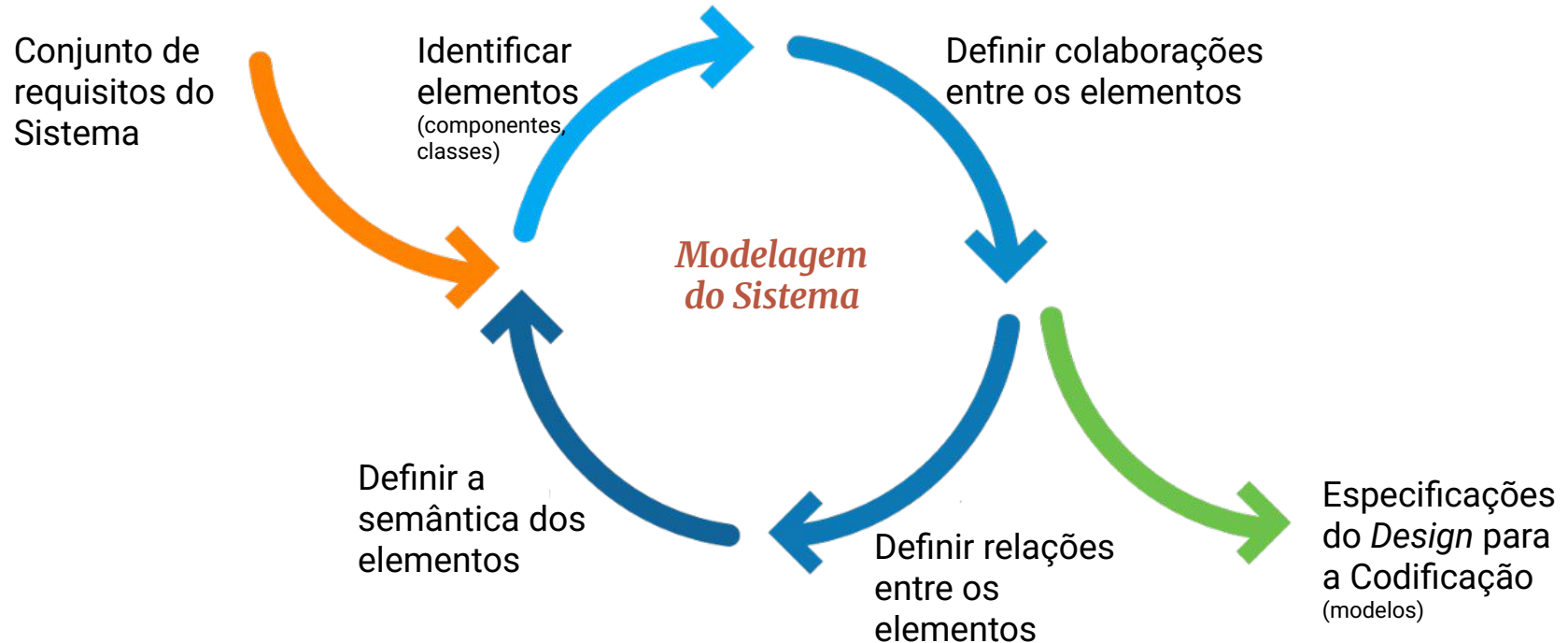
# O Subprocesso de Análise e Projeto (Design)



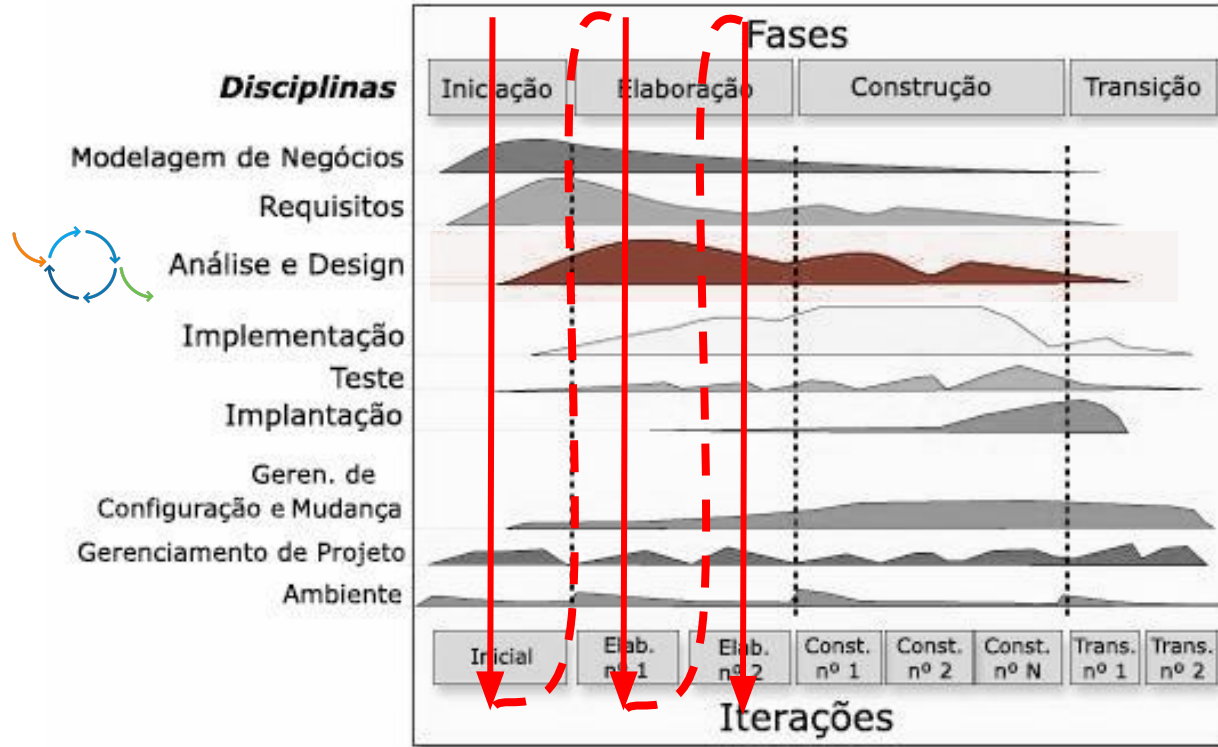
## Objetivo da Análise e do Projeto

- Transformar os requisitos em projeto (design) do sistema de software.
- Identificar os elementos (componentes, módulos, serviços, classes, objetos) que devem ser implementados no sistema
- Criar uma arquitetura robusta do sistema considerando os elementos identificados
- Especificar a implementação dos elementos do sistema.
- **Especificação do sistema = Modelagem**

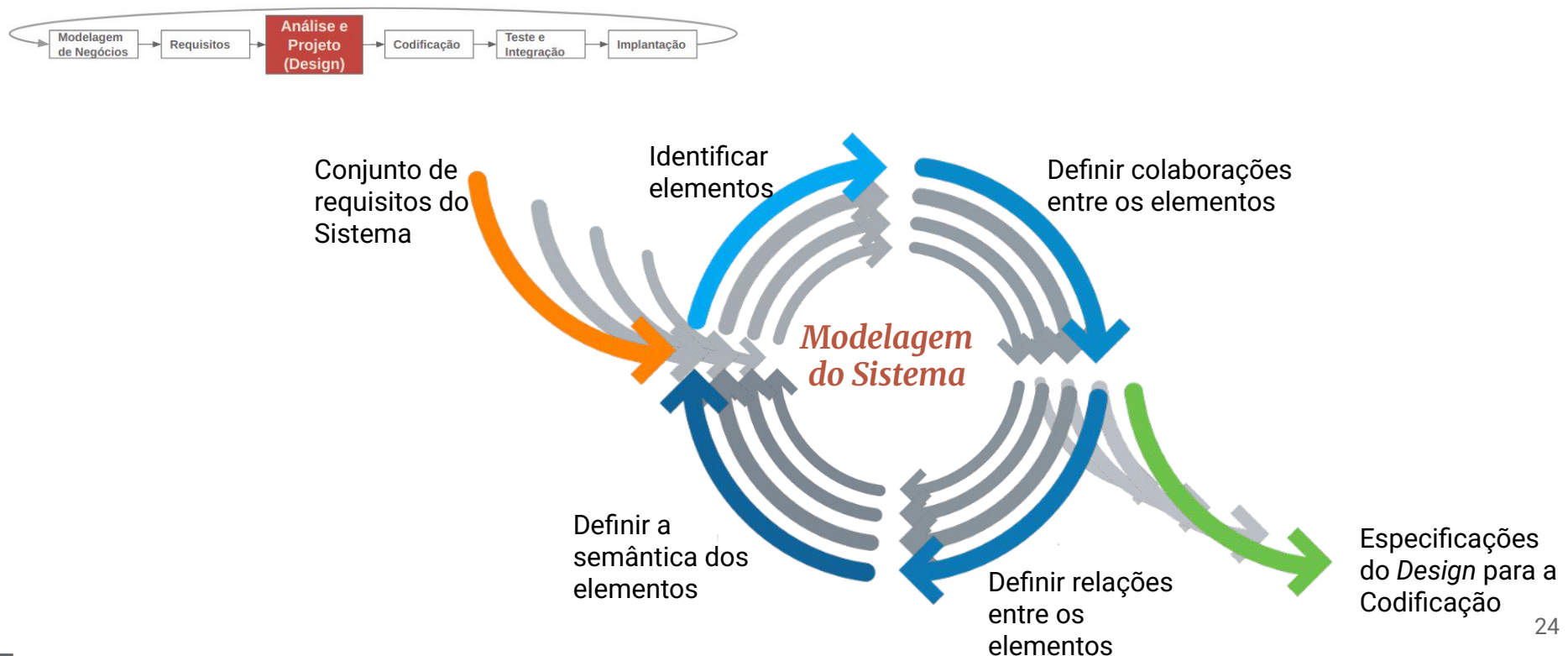
# O Subprocesso de Análise e Projeto (Design)



# Análise e Projeto é Iterativamente Refinado



# O Subprocesso de Análise e Projeto (Design) é Iterativo e Incremental



O Projeto (*Design*) é uma Abstração  
do Software

# Abstrações no Desenvolvimento de Software

+ Abstrato  
- Concreto



- Abstrato  
+ Concreto

Vazio, Desconhecido, Idéia

Modelos Conceituais, Mapas Mentais,  
Modelos de negócios, Requisitos ...

Arquiteturas de Referência, Modelos de  
Referência, Linhas de Produto, Padrões ...

Projeto da Arquitetura (componentes,  
serviços, módulos, interfaces, APIs...)

Projeto de Componentes (Classes, objetos,  
atributos, métodos, interações)

Variáveis, estruturas de dados

código máquina

Código binário

hardware: cpu, disco, memória, rede, ....

Frameworks de  
Desenvolvimento,  
Tecnologias,  
Bibliotecas,  
Linguagens de  
Programação ...

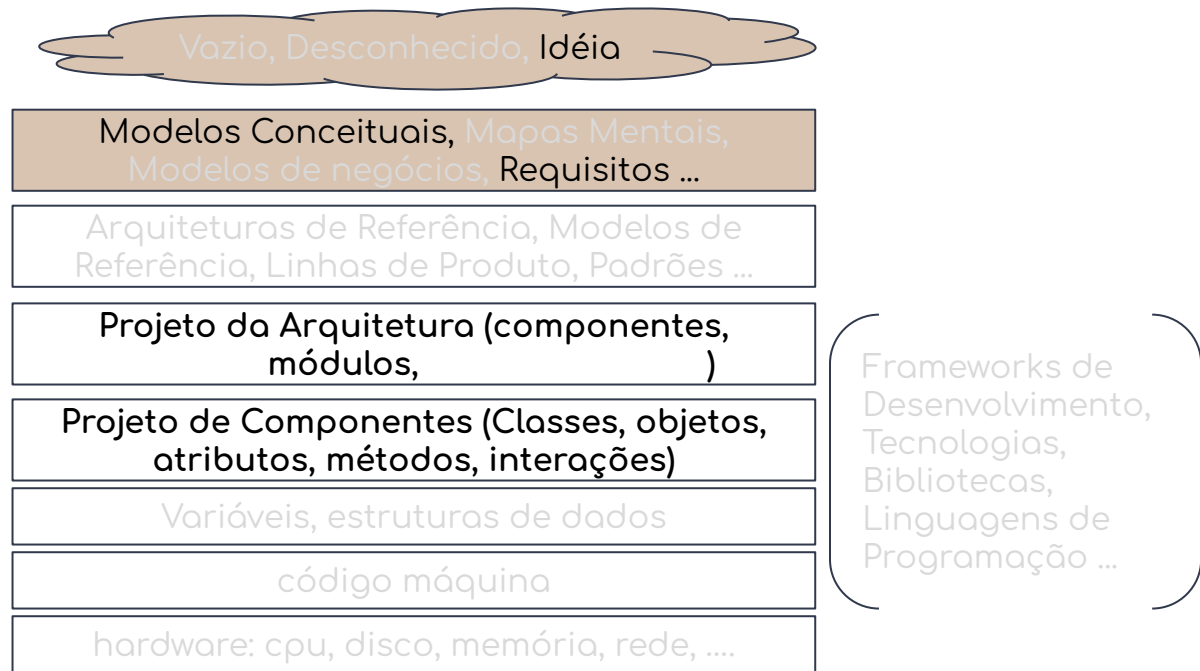
# Abstrações no Desenvolvimento de Software

## Nosso foco em COM221

+ Abstrato  
- Concreto



- Abstrato
- + Concreto



# O Projeto (Design) de Software é Abstrato

## *O Conceito de Abstração*

**Abstrações** do sistema de software **de alto nível** são realizadas durante as **primeiras etapas do desenvolvimento de software** (preliminares à codificação)

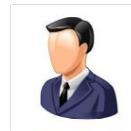
As abstrações são criadas através da seleção dos *aspectos do sistema de software mais relevantes para um determinado propósito*, por exemplo, entendimento do problema a ser resolvido (diminuição de custos, aumento de desempenho, alcançar novos mercados, melhorar a atenção ao cliente), identificação dos requisitos mais importantes, etc.

# O Projeto (Design) de Software é Abstrato

*Dependendo do problema, objetivo, contexto, e stakeholders envolvidos...*

As abstrações *apresentam simplificações do sistema* que descrevem aspectos importantes a serem abordados. Aspectos menos relevantes são deixados para níveis mais concretos.

As abstrações *podem ser úteis ou não* no entendimento dos problemas (ex. o código fonte não é útil para entender os objetivos do cliente; a arquitetura não é útil para entender os métodos dos objetos;



Cliente



Desenvolvedor



Analista

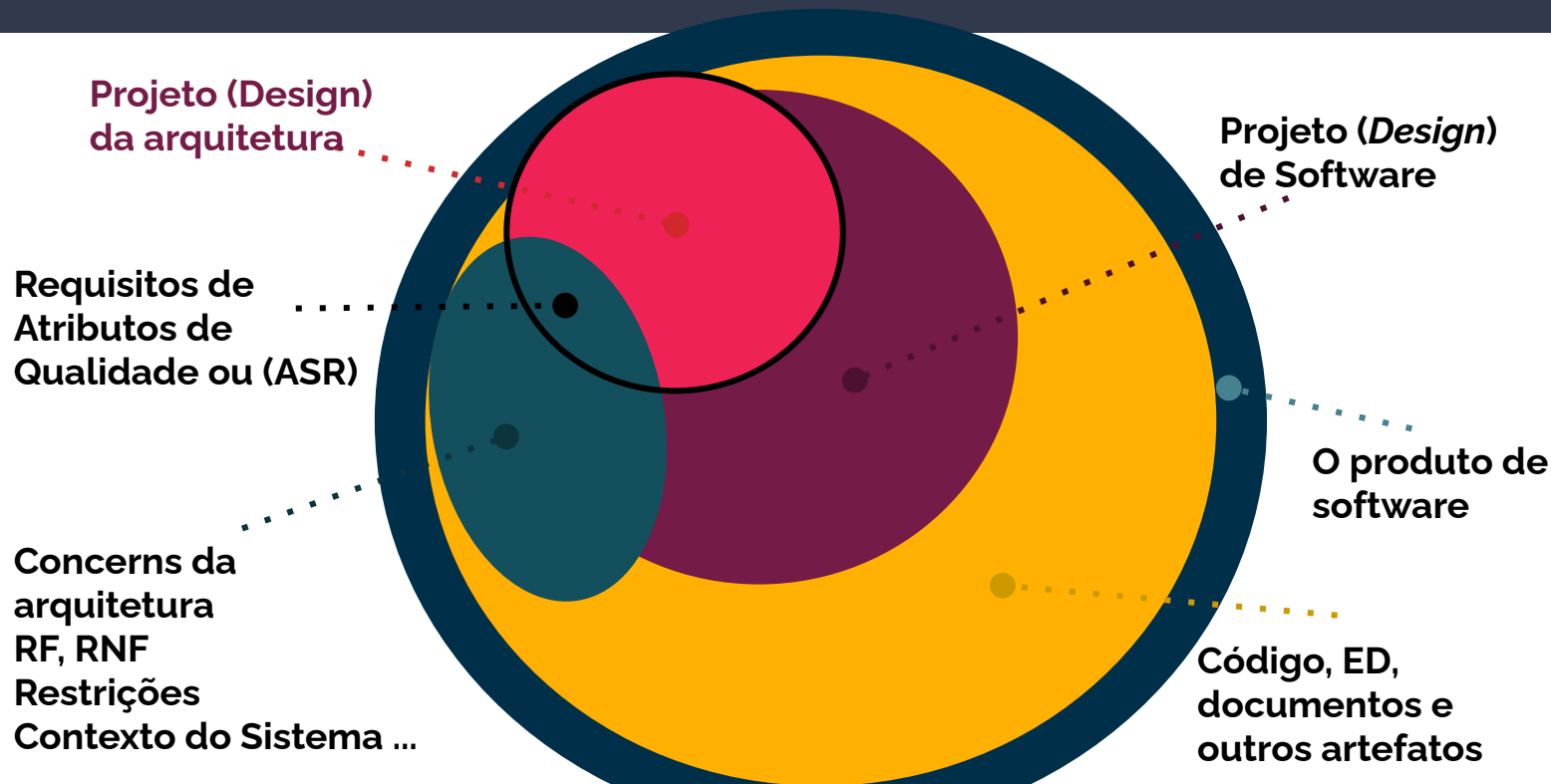


Manager da equipe



Usuários finais

# Diferença entre *Design* de Software Abstrato e Concreto



# Diferença entre *Design* de Software Abstrato e Concreto

Arquitetura é um **projeto (design) abstrato**, cujos elementos são

- Componentes
- Módulos
- Serviços
- Middleware
- Barramentos ...

Projeto de componentes é um **projeto (design) mais concreto**, focando-se em elementos como

- classes de objetos
- métodos de classes
- atributos
- relações entre objetos ...

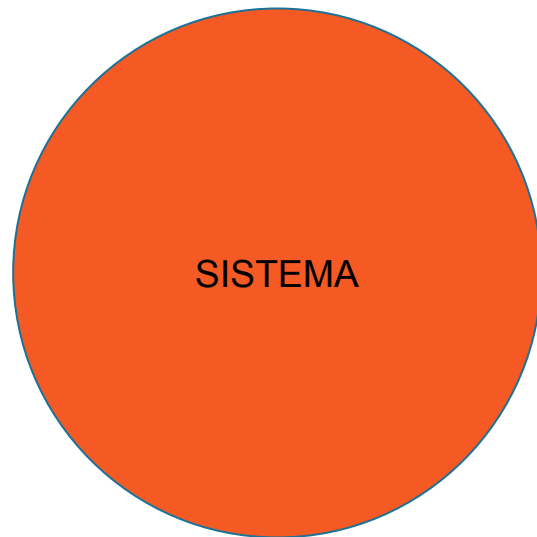
# Diferença entre o *Desing* Arquitetural e o *Design* de Software

- A arquitetura é projeto (design), mas *nem todo projeto (design) pode ser considerado arquitetura*.
- Ou seja, *muitas decisões de projeto (design) não são consideradas* na arquitetura, mas são deixadas para que os programadores ou outros projetistas façam em níveis mais concretos.
- A arquitetura estabelece **RESTRIÇÕES** nas atividades futuras (design de componentes, codificação, teste..), e essas atividades devem produzir **artefatos** (código ou projeto mais detalhados) **conforme** à arquitetura. (Clements et al. 2010).

# Abstração nas Etapas de Análise e Design do Sistema

## *Estabelecer:*

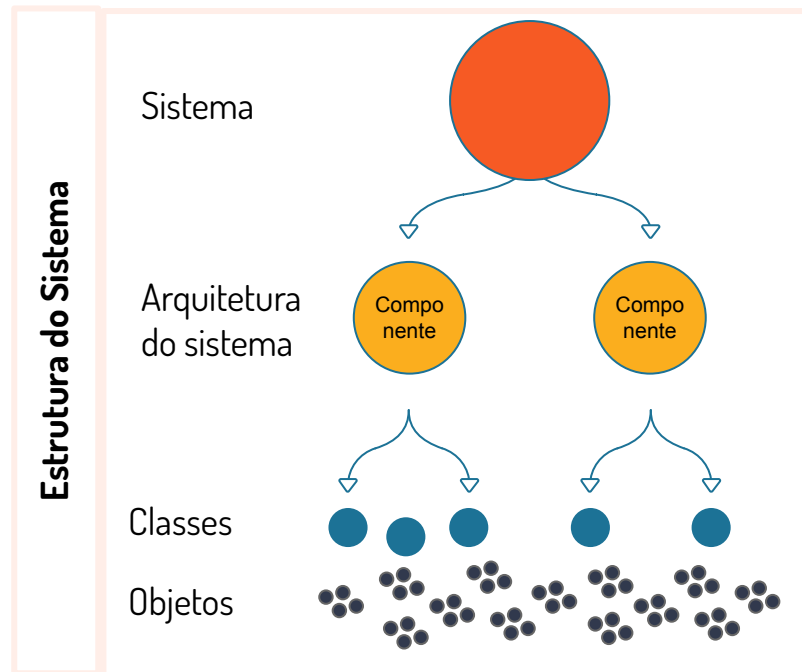
1. **Objetivos/Propósitos do sistema** - Quais problemas o sistema deverá resolver?
2. **Diretrizes e restrições** no sistema que irão guiar e fornecer o contexto para o projeto da arquitetura (p.ex. requisitos de segurança, privacidade, confiabilidade, desempenho..)
3. **Especificar Requisitos Funcionais** - O quê o o sistema deverá executar? Que funcionalidades deverá fornecer?



# Abstração nas Etapas de Análise e Design do Sistema

## *Estabelecer a Estrutura do Sistema:*

1. Identificar os **elementos mais relevantes** para o projeto (design) do sistema de software, incluindo **boas práticas de projeto** e seguindo as diretrizes identificadas na análise da arquitetura
2. **Refinar os elementos** do projeto (design) para produzir especificações (classes, objetos) que possam ser eficientemente implementadas em código.

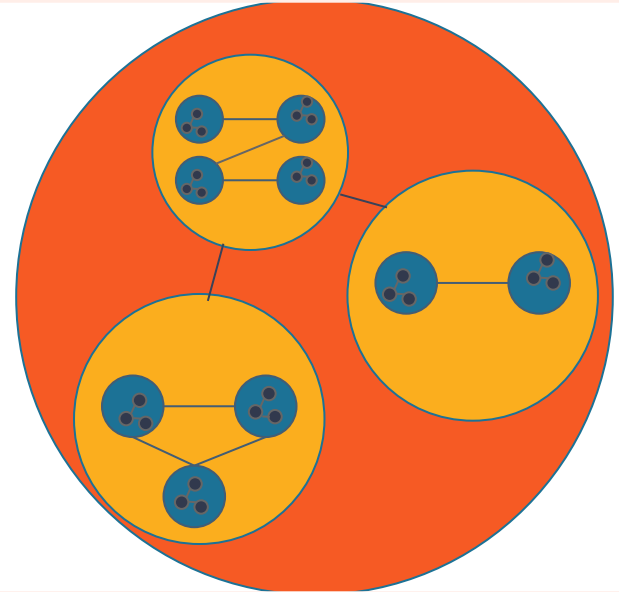


# Abstração nas Etapas de Análise e Design do Sistema

## *Especificar o Comportamento do Sistema*

1. **Alocar requisitos** a elementos mais concretos (p.ex. classes)
2. Especificar interações entre elementos da arquitetura (componentes)
3. Especificar interações entre elementos de design (classes e objetos) para produzir os comportamentos (funcionalidades) desejadas no sistema

Comportamento do Sistema



# Abstração nas Etapas de Análise e Projeto (*Design*)

Nível de  
Abstração

Análise

Projeto (*Design*)

Arquitetura

*Foco:* Estabelecer as **diretrizes e restrições** no sistema que irão guiar e fornecer o contexto para o projeto da arquitetura (p.ex. requisitos de segurança, confiabilidade, desempenho..)

*Foco:* Identificar os **elementos mais relevantes** para o projeto do sistema de software, incluindo **boas práticas de projeto** e seguindo as diretrizes identificadas na análise da arquitetura.

Componente

*Foco:* Entender o conjunto atual de requisitos. Produzir uma versão inicial da solução. **Alocar requisitos** a elementos mais concretos (p.ex. classes)

*Foco:* **Refinar os elementos** do projeto para produzir especificações que possam ser eficientemente implementadas em código.

Perguntas?

# Resumo

# Resumo

- O software é **desenvolvido** através de um **processo de engenharia**
- Existem **diversos PDS** (processos de desenvolvimento de software) que as equipes podem **adotar** e **modificar** dependendo do contexto, ambiente e natureza do software a ser desenvolvido.
- Para participar de uma equipe de desenvolvimento de software, além das habilidades técnicas precisamos adquirir **soft-skills** (boa comunicação, trabalho colaborativo, empatia, compromisso, ética, etc.)
- O **ciclo de vida de um software** (décadas) é bem maior comparado com o ciclo de vida do hardware (anos)

# Resumo

- O Processo Unificado é **iterativo e incremental**, e define **4 fases** e **9 disciplinas**.
- O Processo Unificado define um sub-processo para direcionar a **Análise, o Projeto e a Modelagem** do sistema de software. Este sub-processo também é **iterativo e incremental**
- As atividades de Análise, Projeto e Modelagem podem ser aplicadas para estabelecer a **arquitetura do sistema de software**, como também para especificar os **elementos internos de componentes**.
- O **projeto do sistema de software é abstrato**:
  - O projeto arquitetural é definido em um nível de abstração maior
  - O projeto interno de componentes é mais concreto.

# Paradigma de Desenvolvimento de Software OO

## PRINCÍPIOS

### Abstração

Módulos, componentes, classes.

### Encapsulamento

Ocultamento de informações e operações, Facilidade de mudança



### Herança

Reuso de estrutura de dados e de comportamento . Coesão.

### Polimorfismo

Flexibilidade na definição de métodos. Facilidade de evolução (incremento)

**MODELAGEM** - Diagramas UML (Unified Modeling Language)

**PROCESSO DE DESENVOLVIMENTO DE SOFTWARE** - Etapas de Análise, Design e Implementação

# Referências

Capítulo 2. **Processos de software**; e Capítulo 5. **Modelagem de sistemas**. do Livro: Ian Sommerville. Engenharia de Software. 9a Ed. 2011.

Capítulo 6. **Process**. Livro: Grady Booch, robert A. Maksymchuk, Michael W. Engle, Bobbi J. Young, Jin Conallen, Kelli A.Houston. Object-Oriented Analysis and Design with Applications. 3rd Ed. 2007.

Capítulo 1. **What is software architecture?** Livro: Len Bass, Paul Clements, Rick Kazman. Software Architecture in Practice. 3rd Ed. 2013.

[Paper] P. Abrahamsson, M. A. Babar and P. Kruchten, "**Agility and Architecture: Can They Coexist?**," in IEEE Software, vol. 27, no. 2, pp. 16-22, March-April 2010.